AEP-X — Operational Manual

Running, verifying, and troubleshooting the live reference implementation — with UML use case and sequence diagrams

Companion to Handoff, ADLC-Plan, Instructional-Manual, Microservices Guide, SOA-Architecture

Prepared 8 July 2026

Covers 14 core services · Connector Bus · 10 connector categories · 100 connectors

Contents

1. System Overview

The reference implementation runs as **29 containers** under one `docker-compose.yml`:

- **Tier 1 — Foundational (synchronous):** Identity, Trust, Registry, Memory, Cache (L0–L5), Discovery — plus the API Gateway that fronts everything.
- **Tier 2 — Orchestration (event-driven):** Workflow, Safety, Governance, Knowledge, Verification, Cost Optimiser, ML Integration.
- **SOA layer:** the Connector Bus plus 10 category connector services carrying **100 catalogued connectors** (4 specialized, 96 governed stubs — see Connector-Catalogue.md).
- **Infrastructure:** PostgreSQL (pgvector), Redis, Neo4j, Kafka (KRaft).

Two invariants govern every runtime path (governance/book-of-laws.md): **Law 2** — no external call executes before a trust evaluation, enforced structurally at the Connector Bus; **Law 8** — every event is audited by the Governance Engine, which consumes every Kafka topic unconditionally.

2. UML Use Case Diagram

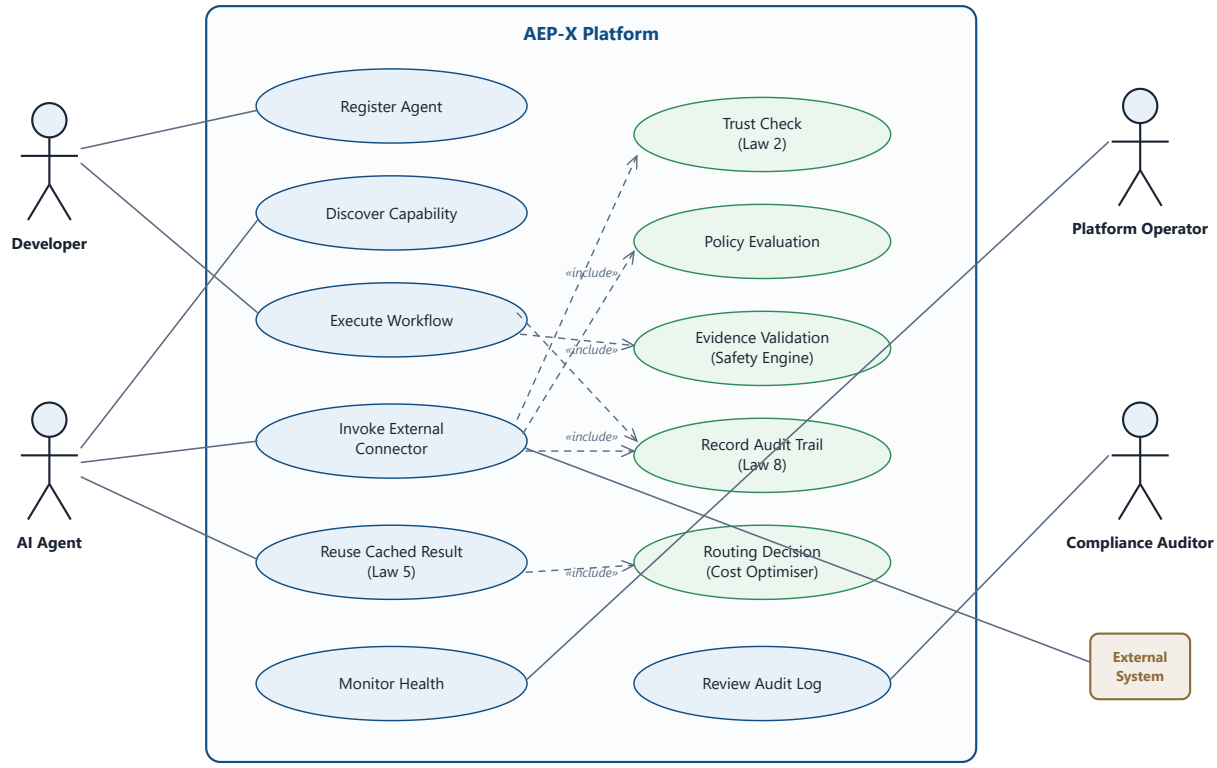

Fig. 1 — Use case diagram. Green use cases are governance-mandated inclusions: invoking any external connector *always* includes the trust check, policy evaluation, and audit recording — they are not optional paths.

BEHAVIOUR

3. UML Sequence Diagrams

The three load-bearing runtime flows, as verified live on 8 July 2026

3.1 Connector Bus mediation (SOA layer — Laws 2 & 8 enforced structurally)

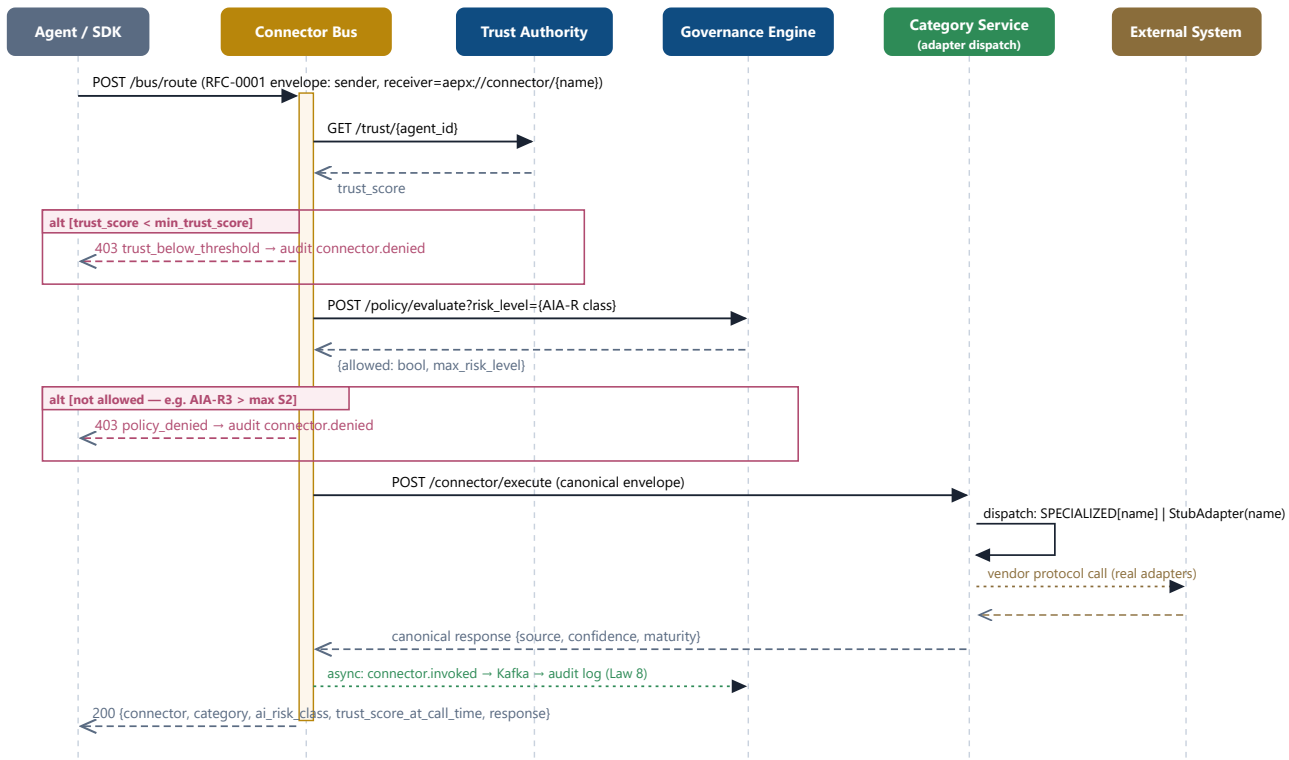

Fig. 2 — Connector Bus mediation. Both denial paths were exercised live: `opcua/gov-pay` (AIA-R3) exit at the policy gate; `sap/moodle` (min trust 60 vs. stub score 50) exit at the trust gate.

3.2 Workflow execution with event choreography (Tier 2 — async, Kafka)

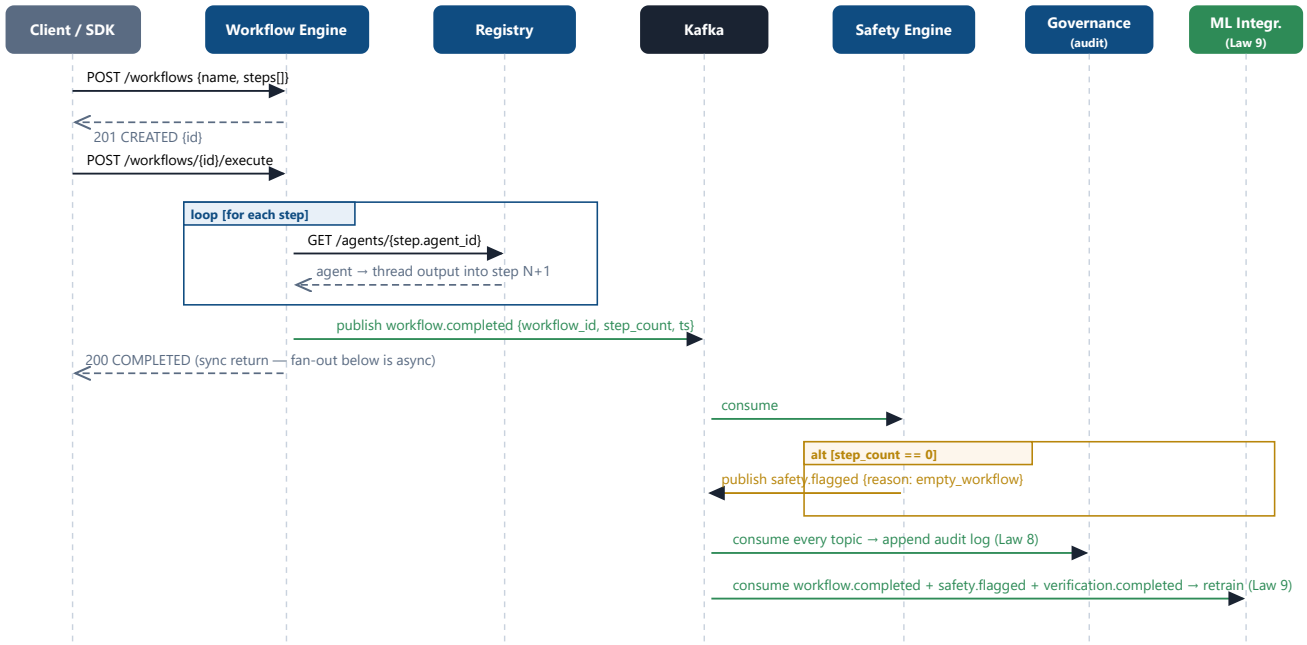

Fig. 3 — Event choreography. Governance is a passive listener — 100% audit coverage without any service having to call it. Consumers use reconnect loops (see §6, defect 3) so a broker restart never silently kills the chain again.

3.3 Cache-first routing (Law 5 — the LLM is the last resort)

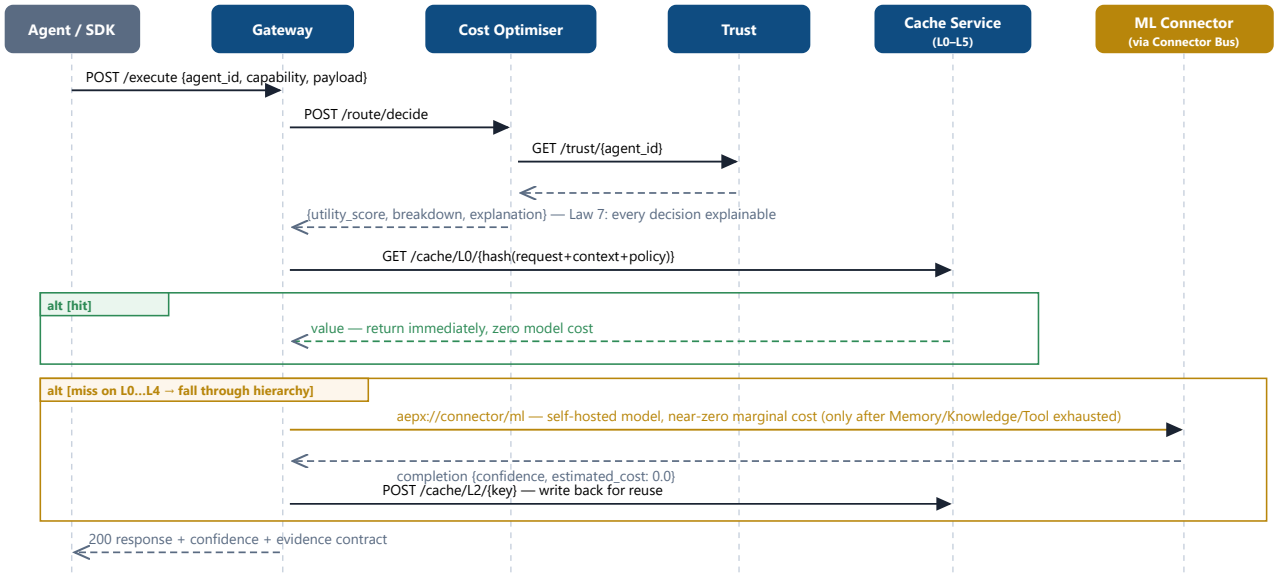

Fig. 4 — Cache-first execution (RFC-0005/0008). The Cost Optimiser's `/route/decide` is the single synchronous Tier-2 exception (<20 ms budget); L5 federation-cache writes additionally require a Governance policy check.

OPERATIONS

4. Operations Runbook

Start, stop, verify — the day-2 procedures for the running stack

4.1 Start / stop

```
# start everything (from the repo root)
docker compose up -d --build

# SOA layer only, after trust + governance are healthy (SOA-Architecture.md §5 ordering)
docker compose up -d trust governance
docker compose up -d --build connector-bus connector-enterprise connector-productivity \
  connector-devtools connector-aiplatform connector-data connector-messaging \
  connector-industrial connector-cloud connector-government connector-education

# stop (containers persist)          # stop and remove (data volumes preserved)
docker compose stop                  docker compose down

# full reset including data volumes — Postgres re-runs schemas/sql/*.sql on next start
docker compose down -v
```

4.2 Port map

Port	Service	Port	Service
8000	Gateway (aggregate `/health`)	8020	Connector Bus
8001	Identity	8021	connector-enterprise (14)
8002	Trust	8022	connector-aiplatform (14)
8003	Registry	8023	connector-devtools (11)
8004	Memory	8024	connector-productivity (11)
8005	Cache (L0–L5)	8025	connector-data (16)
8006	Discovery	8026	connector-messaging (8)
8007	Workflow Engine	8027	connector-industrial (8)
8008	Safety Engine	8028	connector-cloud (10)
8009	Governance (`/audit`, `/policy/evaluate`)	8029	connector-government (4)
8010	Knowledge	8030	connector-education (4)
8011	Verification	5432	PostgreSQL (pgvector/pgvector:pg16)
8012	Cost Optimiser	6379	Redis
8013	ML Integration	7474/7687	Neo4j (HTTP / Bolt)
8080	**Console — the LLM/ML box web GUI** (chat via Connector Bus; file/folder/image/video upload)	9092	Kafka (KRaft)

4.3 Health verification

```
curl http://localhost:8000/health     # 14 core services, aggregated
curl http://localhost:8020/health     # bus: expect "connector_count": 100, 10 categories
curl http://localhost:8009/audit      # audit log — grows with every workflow/safety event
docker compose ps                     # 29 containers; postgres/redis/neo4j report (healthy)
```

5. Smoke Test Procedure

Run after any rebuild. Every step below was executed successfully against the live stack on 8 July 2026.

```
# 1. Register an agent
AGENT_ID=$(curl -s -X POST http://localhost:8003/agents \
  -H "Content-Type: application/json" -d '{"name":"smoke-agent"}' | jq -r .id)

# 2. Create + execute a workflow → expect status COMPLETED
WF_ID=$(curl -s -X POST http://localhost:8007/workflows -H "Content-Type: application/json" \
  -d "{\"name\":\"smoke\",\"steps\":
[{\"capability\":\"lesson_generation\",\"agent_id\":\"$AGENT_ID\"}]}" | jq -r .id)
curl -s -X POST http://localhost:8007/workflows/$WF_ID/execute | jq .status

# 3. Within ~2s the event must appear in the audit log (Law 8)
curl -s http://localhost:8009/audit | jq '.[-1].topic'    # → "workflow.completed"

# 4. Connector Bus — specialized adapter (expect Jane Doe)
curl -s -X POST http://localhost:8020/bus/route -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/'$AGENT_ID'","receiver":"aepx://connector/salesforce","payload":
{"op":"lookup_contact"}}' | jq .response.result.name

# 5. Connector Bus — governance gates (expect HTTP 403 on both)
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8020/bus/route \
  -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/x","receiver":"aepx://connector/opcua","payload":{}}'   # policy
gate
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8020/bus/route \
  -H "Content-Type: application/json" \
  -d '{"sender":"aepx://agent/x","receiver":"aepx://connector/sap","payload":{}}'     # trust
gate

# 6. Cost Optimiser — explainable routing decision (Law 7)
curl -s -X POST http://localhost:8012/route/decide -H "Content-Type: application/json" \
  -d
'{"agent_id":"'$AGENT_ID'","capability":"x","candidate_route":"cache","estimated_latency_ms":5
}' | jq .explanation
```

> **Pass criteria:** step 2 returns `COMPLETED`; step 3 shows the event within 2 seconds; step 4 returns `Jane Doe`; step 5 returns `403` twice; step 6 returns an explanation string containing the utility score.

6. Troubleshooting Guide

The first three entries are real defects found and fixed during the 7–8 July deployment — kept here because their symptoms will recur in any fresh environment that repeats the original configuration.

Symptom	Root cause	Fix
`uvicorn: executable file not found in $PATH` at container start; image builds "succeed" but pip layers are ~4 kB	Disk full during build — containerd writes truncated layers silently (`input/output error` in daemon logs)	Free disk space (Docker's VHDX lives under `AppData\Local\Docker`), then `docker compose build --no-cache`
Postgres exits (3): `extension "vector" is not available`	Stock `postgres:16` doesn't ship pgvector, which `002_v2_additions.sql` requires	Use `pgvector/pgvector:pg16` (already in docker-compose.yml). If a stale anonymous volume shows *collation mismatch* warnings: `docker compose rm -sfv postgres && docker compose up -d postgres`
Workflows complete but `GET /audit` stays empty; Kafka topic exists but has no consumers	Consumer/producer created once at import lost the startup race against the broker and died silently (no retry)	Fixed in source: reconnect loops (Governance, Safety) + lazy producers (Workflow, Safety). If it recurs: `docker compose restart workflow safety governance ml-integration` and check only `MainThread` isn't alone via `docker exec … python -c "import threading; print(threading.enumerate())"`
Kafka exits at preflight: `Cluster ID string … is not a valid UUID`	KRaft requires a base64-encoded 16-byte UUID (22 chars), not an arbitrary string	Generate one: `python -c "import uuid,base64; print(base64.urlsafe_b64encode(uuid.uuid4().bytes).decode().rstrip('='))"` → set as `CLUSTER_ID`
Bus returns `403 trust_below_threshold`	Agent trust (stub: 50) below the connector's `min_trust_score` (e.g. SAP/education: 60)	Working as designed (Law 2). Unlocks when Trust Authority implements the RFC-0002 5-component formula — or is a per-connector governance decision in the catalogue
Bus returns `403 policy_denied` for industrial/government connectors	AIA-R3 exceeds Governance's seed `max_risk_level: S2`	Working as designed. Raising the ceiling is an explicit governance decision (`_POLICIES` in governance service), never a code workaround

Bus `/health` shows `connector_count: 0`	`catalogue.json` not mounted at `/app/catalogue.json`	Check the `volumes:` entry for connector-bus and every connector-* service in docker-compose.yml
Console shows "⚠️ **Could not reach the platform**"; `docker version` / `docker ps` hang or return a 500 from the Linux engine pipe	Docker Desktop's daemon has wedged — on memory-constrained hosts (<8 GB RAM) this recurs under load once ~30 containers plus a loaded model are running, especially if Docker Desktop's **Kubernetes** feature is also enabled (it adds ~18 unused system containers: etcd, kube-apiserver, coredns, etc.)	Restart the daemon: stop all `*docker*` processes, `wsl --shutdown`, relaunch Docker Desktop, wait ~40s, then `docker compose ps` — every service has `restart: unless-stopped` so the whole stack self-heals with no further commands. If this recurs, disable Kubernetes in Docker Desktop Settings → Kubernetes (or set `"KubernetesEnabled": false` in `%APPDATA%\Docker\settings-store.json` while Docker Desktop is stopped) — this alone cut a live chat response from ~20s to ~6s on a 7.7 GB-RAM host by freeing an entire unused control-plane's worth of memory.
First chat reply after any Ollama restart returns `maturity: specialized_degraded` with a "model unreachable" message	Cold start — Ollama must load the model into RAM before its first inference, which can exceed the adapter's timeout (25s) on a constrained host	Working as designed (graceful degradation, not a bug). Warm it manually: `curl -X POST http://localhost:11434/api/generate -d '{"model":"llama3.2:1b","prompt":"hi","stream":false}'`, then retry — subsequent calls return in 3–7s

7. Backup & Restore

- **What to back up:** the repo itself is the system of record — all images rebuild from Dockerfiles; base images re-pull from Docker Hub. Runtime state that matters lives in Postgres volumes only once real pilot data exists.
- **Current backups (8 July 2026):** `OneDrive\Backups\AEP-X\aepx-core-full-2026-07-08.tar.gz` (complete repo) and `Docker-Backup-2026-07-08\` (build definitions + restore README). Cloud sync engages when the OneDrive client is signed in.
- **Restore:** extract the tarball → `docker compose up --build`. Postgres re-initialises from `schemas/sql/` on first boot with an empty volume.
- **Do not back up Docker's binary image data** (`docker_data.vhdx`) — it is ~20 GB of rebuildable artifacts, and images built during a disk-full incident are corrupted anyway (see §6, defect 1).

8. Connector Operations

8.1 Promoting a stub to a real integration

1. Write an adapter class in the category service's `app/adapters.py` implementing `execute(payload) → dict`, and register it: `SPECIALIZED["notion"] = NotionAdapter()`.
2. Store credentials in Vault/env — never in the catalogue or code.
3. Update the connector's `maturity` to `specialized` in `connectors/catalogue.json` (single source of truth) and regenerate the SQL/docs.
4. For AIA-R2+: record the assurance-tier sign-off in `governance/decisions/` *before* first live traffic (SOA-Architecture.md §4).
5. `docker compose up -d --build connector-<category>` — nothing else redeploys.

8.2 Adding a brand-new connector

Add one entry to `connectors/catalogue.json` and restart the bus + owning category service. It is immediately routable as a governed stub — trust-checked, policy-gated, audited — with no code change. That is the point of the data-driven routing table.

8.3 The two governance gates (do not bypass)

> A `403` from the bus is a **feature**. The trust gate (Law 2) and the policy ceiling protect exactly the connectors — industrial control systems, government services, HR/finance platforms — where an ungoverned agent call does real-world damage. Raise an agent's trust or the risk ceiling through governance records, never by editing the check out of the bus.

9. Operational Definition of Done

- All 29 containers up; `postgres`, `redis`, `neo4j` report `(healthy)`.
- Gateway `/health`: all 14 services `ok`. Bus `/health`: `connector_count: 100`, 10 categories.
- Smoke test (§5) passes all six criteria, including both 403 governance gates.
- A workflow execution appears in `GET /audit` within 2 seconds (choreography chain alive).
- Backups current (§7) and the restore path tested at least once per release.